

Résolution d'Équations Différentielles par Réseaux de Neurones

Implémentation PyTorch des méthodes PINN et DeepRitz

Ait Abbou Soufiane, Paul Rivaud, Timoté Missaye, Pierre Kairo

6 juin 2026

Plan de la présentation

- ➊ Introduction & méthode PINN *(cette partie)*
- ➋ Méthode Deep Ritz & notre approche du problème
- ➌ Difficultés rencontrées & présentation du code
- ➍ Résultats, optimisation des hyperparamètres & conclusion

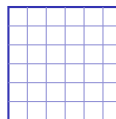
- Les **équations aux dérivées partielles (EDP)** modélisent d'innombrables phénomènes : mécanique, thermique, finance. . .
- Mais très peu d'EDP admettent une **solution analytique** : on les résout **numériquement**.

Problématique

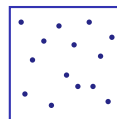
Peut-on résoudre une EDP en exploitant la puissance des **réseaux de neurones**, *sans recourir à un maillage* ?

Les méthodes classiques et leurs limites

- Méthodes traditionnelles : **différences finies, éléments finis.**
- Principe : **discrétiser** le domaine par un **maillage**.
- Limites :
 - coût explosif en **grande dimension** ;
 - géométries complexes, **remaillage**.



Maillage



PINN

méthode classique vs sans maillage

Problème modèle — équation de Poisson 1D

$$-u''(x) = f(x) \text{ sur }]0, 1[, \quad u(0) = u_0, \quad u(1) = u_1.$$

Pourquoi des réseaux de neurones ?

1. Approximation universelle (Cybenko, 1989)

Un réseau à **une seule couche cachée** et activation continue approxime **uniformément** toute fonction continue. On cherche donc la solution dans l'espace des réseaux :

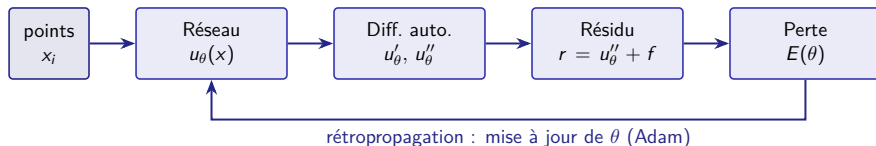
$$u_{\theta}(x) = \sum_{i=1}^n c_i \sigma(w_i x + b_i).$$

2. Intégration de la physique (approche PINN)

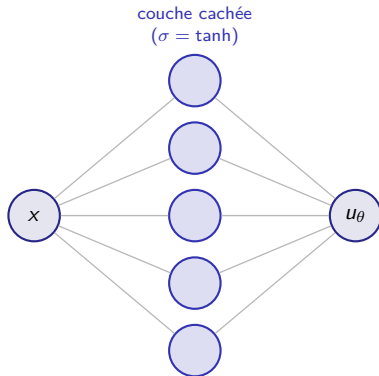
On incorpore l'EDP et les conditions aux bords **directement** dans l'apprentissage. Cadre **non supervisé** : aucune donnée étiquetée, on minimise le **résidu** de l'EDP.

Principe des PINN (Raissi, Perdikaris & Karniadakis, 2019)

- Le réseau u_θ est la solution approchée ; ses dérivées u'_θ , u''_θ sont obtenues par **différentiation automatique**.
- On transforme l'EDP en un **problème d'optimisation**.



Architecture du réseau



- Espace des réseaux ($\theta \in \mathbb{R}^{3n}$) :
$$V_{nn} = \left\{ x \mapsto \sum_{i=1}^n c_i \sigma(w_i x + b_i) \right\}$$
- Activation $\sigma = \tanh$, donc $\sigma'(x) = 1 - \tanh^2(x)$.
- $C^\infty \Rightarrow u_\theta \in C^2$ et u_θ'' **calculable** par autodiff.

La fonction de perte physique

La fonctionnelle d'énergie mesure à quel point u_θ viole l'EDP et les bords :

$$E(u_\theta) = \int_{\Omega} |u''_\theta(x) + f(x)|^2 dx + (u_\theta(0) - u_0)^2 + (u_\theta(1) - u_1)^2.$$

En pratique (collocation sur N points x_i) :

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N} \sum_{i=1}^N (u''_\theta(x_i) + f(x_i))^2}_{\mathcal{L}_{\text{PDE}} : \text{résidu de l'EDP}} + \underbrace{(u_\theta(0) - u_0)^2 + (u_\theta(1) - u_1)^2}_{\mathcal{L}_{\text{BC}} : \text{conditions aux bords}}$$

- u^* solution exacte $\iff E(u^*) = 0$, d'où $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$.
- Minimisation par **descente de gradient (Adam)** ; $\nabla_{\theta} \mathcal{L}$ obtenu par **rétropropagation**.

Conclusion de partie & transition

- Les PINN reformulent une EDP en un **problème d'optimisation** : le réseau apprend à **annuler le résidu** de l'équation.
- Atouts : **sans maillage, non supervisé**, adaptés aux grandes dimensions et aux problèmes inverses.

Transition

Place à la **méthode Deep Ritz** (formulation variationnelle) et à notre **implémentation PyTorch**.